

EJERCICIO 27

Archivo cola.rs

Este archivo define una estructura de cola genérica Cola<T, const SIZE: usize>:

1. Estructura Cola:

- Define una cola circular con un tamaño fijo SIZE y un tipo genérico T.
- Los campos first, last, y len mantienen el estado de la cola:
 - first: índice del primer elemento.
 - last: índice donde se añadirá el próximo elemento.
 - len: número de elementos actuales en la cola.

2. Constructor new:

- Inicializa una cola vacía, configurando los índices en 0 y llenando el array a con valores predeterminados de T.

3. Método enqueue:

- Añade un elemento al final de la cola, verificando primero si hay espacio. Si la cola está llena, devuelve un error.

4. Método dequeue:

- Elimina y devuelve el primer elemento de la cola, ajustando el índice first y reduciendo len.

Archivo main.rs

Este archivo es el punto de entrada y maneja la evaluación de expresiones matemáticas desde un archivo de texto.

1. Función apply_operator:

- Aplica un operador (e.g., +, -, *, /, sqrt) sobre operandos extraídos de una pila operands.

2. Función eval:

- Evalúa expresiones matemáticas.
- Usa una cola para almacenar tokens de la expresión, y dos pilas: una para operadores y otra para operandos.

- Al finalizar la evaluación, devuelve el resultado o un error si hay un problema de sintaxis.

3. Función main:

- Lee expresiones desde expressions.txt y evalúa cada una, imprimiendo el resultado.
- Incluye pruebas (tests) para evaluar casos específicos, verificando la corrección del código.

Archivo pila.rs

Define una estructura de pila genérica `Pila<T, const SIZE: usize>`:

1. Estructura Pila:

- Implementa una pila básica con tamaño `SIZE` y tipo genérico `T`.
- `cima` indica la posición donde se añadirá el próximo elemento.

2. Método push:

- Añade un elemento en la cima de la pila, devolviendo error si la pila está llena.

3. Método pop:

- Extrae el elemento en la cima, devolviendo error si la pila está vacía.

4. Método is_empty:

- Verifica si la pila está vacía.

Archivo read_file.rs

Define la función `read`, que lee un archivo de texto línea por línea:

1. Función read:

- Abre un archivo y lee cada línea, eliminando espacios en blanco y errores de conversión, retornando un vector de `String`.

Archivo tokens.rs

Define una función `next_token` para extraer tokens de una expresión:

1. Función next_token:

- Toma una cadena `s` y un índice `start`.
- Devuelve una tupla con los índices de inicio y final de un token, o `None` si no hay más tokens.

¿QUÉ SE HACE EN EL MAIN?

El archivo main.rs se encarga de gestionar el flujo de la aplicación, especialmente en la evaluación de expresiones matemáticas. A continuación, profundizo en cada parte del archivo.

1. Importaciones y Módulos

rust

Copiar código

```
use cola::Cola;
```

```
use pila::Pila;
```

```
use tokens::next_token;
```

```
mod tokens;
```

```
mod cola;
```

```
mod pila;
```

```
mod read_file;
```

Aquí se importan las estructuras y funciones de los módulos cola, pila y tokens, que se utilizan en la evaluación de expresiones. También se incluyen los módulos tokens, cola, pila, y read_file para que Rust los reconozca como partes del proyecto.

2. Definición de Constantes

rust

Copiar código

```
const ERR_MSG: &str = "Incorrect expression";
```

Esta constante define un mensaje de error genérico, que se usa para informar al usuario cuando una expresión no es válida.

3. Función apply_operator

rust

Copiar código

```
fn apply_operator(operator: &str, operands: &mut Pila<f64, 25>) -> Result<(),  
String> {
```

```

let Ok(right_operand) = operands.pop() else {
    return Err(String::from(ERR_MSG));
};

if operator == "sqrt" {
    if operands.push(right_operand.sqrt()).is_err() {
        return Err(String::from(ERR_MSG));
    }
} else {
    let Ok(left_operand) = operands.pop() else {
        return Err(String::from(ERR_MSG));
    };

    let result: f64 = match operator {
        "+" => left_operand + right_operand,
        "-" => left_operand - right_operand,
        "*" => left_operand * right_operand,
        "/" => {
            if right_operand == 0.0 {
                return Err(String::from(ERR_MSG));
            }
            left_operand / right_operand
        }
        _ => { return Err(String::from(ERR_MSG)); }
    };

    if operands.push(result).is_err() {
        return Err(String::from(ERR_MSG));
    }
}

Ok(())

```

}

Esta función aplica un operador específico sobre operandos extraídos de la pila operands. Funciona de la siguiente forma:

1. **Obtención del Operando Derecho:** Intenta extraer el último valor de la pila operands (el operando derecho) con pop().
 - Si falla, retorna un error indicando que la expresión es incorrecta.
2. **Operador sqrt:**
 - Si el operador es "sqrt", calcula la raíz cuadrada del operando derecho y lo coloca de nuevo en la pila. Si push falla, retorna un error.
3. **Operadores Binarios:** Para operadores como "+", "-", "*", y "/", extrae un segundo operando (el izquierdo) de la pila y aplica el operador:
 - Si el operador es "/" y el operando derecho es 0, retorna un error para evitar división por cero.
 - Finalmente, inserta el resultado en la pila.

4. Función eval

rust

Copiar código

```
fn eval(e: String) -> Result<f64, String> {  
    let mut tokens: Cola<&str, 50> = Cola::new();  
    let mut start = 0;  
    while let Some((begin, end)) = next_token(&e, start) {  
        if tokens.enqueue(&e[begin..=end]).is_err() {  
            return Err(String::from(ERR_MSG));  
        }  
        start = end + 1;  
    }  
}
```

```
let mut operands: Pila<f64, 25> = Pila::new();  
let mut operators: Pila<&str, 25> = Pila::new();
```

```

while let Ok(token) = tokens.dequeue() {
  match token {
    "(" => { /* Ignored */ }
    ")" => {
      let Ok(operator) = operators.pop() else {
        return Err(String::from(ERR_MSG));
      };
      if apply_operator(operator, &mut operands).is_err() {
        return Err(String::from(ERR_MSG));
      }
    }
    "+" | "-" | "*" | "/" | "sqrt" => {
      if operators.push(token).is_err() {
        return Err(String::from(ERR_MSG));
      }
    }
    _ => {
      let Ok(operand) = token.parse::<f64>() else {
        return Err(String::from(ERR_MSG));
      };
      if operands.push(operand).is_err() {
        return Err(String::from(ERR_MSG));
      }
    }
  }
}

match operands.pop() {
  Ok(result) => {

```

```

    if operands.is_empty() {
        Ok(result)
    } else {
        Err(String::from(ERR_MSG))
    }
}

Err(_) => Err(String::from(ERR_MSG)),
}
}

```

La función eval evalúa una expresión matemática en formato de cadena utilizando una cola y dos pilas. Funciona de la siguiente forma:

1. Tokenización:

- Usa next_token para extraer tokens (como operadores y operandos) y los coloca en una cola.
- Si enqueue falla, se retorna un error, ya que esto indica una cola llena o un problema en la expresión.

2. Pilas para Operadores y Operandos:

- operands: pila para operandos numéricos.
- operators: pila para operadores.

3. Evaluación de Tokens:

- Itera sobre cada token en la cola:
 - "(": se ignora, ya que en esta implementación no afecta el orden.
 - ")": al encontrar un paréntesis de cierre, extrae un operador de operators y aplica la operación.
 - +, -, *, /, sqrt: son operadores, así que los agrega a operators.
 - Operandos numéricos: los convierte a f64 y los coloca en operands.

4. Resultado Final:

- Extrae el último valor de operands para devolver el resultado final.

- Si aún hay elementos en operands, significa que la expresión fue incorrecta, y retorna un error.

5. Función main

rust

Copiar código

```
fn main() {  
    let expressions = read_file::read("expressions.txt").unwrap();  
  
    for e in expressions {  
        println!("{}", eval(e).unwrap());  
    }  
}
```

La función main lee expresiones desde el archivo expressions.txt usando read_file::read. Luego, evalúa cada expresión con eval y muestra el resultado.

6. Módulo de Pruebas

rust

Copiar código

```
#[cfg(test)]  
mod tests {  
    use super::read_file;  
  
    #[test]  
    fn test_0() {  
        let expressions = read_file::read("expressions.txt").unwrap();  
        assert_eq!(super::eval(expressions[0].clone()), Ok(28.0));  
    }  
    // Pruebas adicionales para diferentes casos...  
}
```

La sección de pruebas incluye varias pruebas unitarias para validar el comportamiento de la función eval con distintos casos. Cada prueba compara el resultado de eval con un valor esperado, evaluando expresiones de expressions.txt.